Práctica 2: Sockets en Python

Sistemas Distribuidos

Ejercicios

1. Probar sockets:
   - Implemente un programa cliente/servidor que comunique dos procesos con los protocolos TCP y UDP.
   - Explique las diferencias existentes entre ambas implementaciones.
   - ¿Qué ocurre si el cliente se inicia antes que el servidor en ambos casos (TCP y UDP)?
2. Cree un programa cliente y un programa servidor en Python. El programa cliente enviará un fichero (.pdf) al servidor, de acuerdo al siguiente flujo de interacción:
   - El programa cliente establecerá una conexión con el servidor y solicitará enviarle un fichero al servidor, especificando el nombre del fichero.
   - El servidor, recibida la petición del cliente, deberá responder afirmativamente. Sin embargo, si ya existe un fichero en el servidor con el mismo nombre del fichero que indica el cliente, el servidor, debe consultar al cliente si desea sobrescribir el fichero existente. En caso afirmativo, el servidor sobrescribiría el fichero. En caso negativo, se cancela la transferencia y se cierra la conexión.
   - El cliente, una vez recibida la respuesta afirmativa, debe enviar el fichero al servidor, quien debe recibirlo y crear el fichero en su directorio de trabajo.
   - El servidor debe notificar al cliente la recepción del fichero.
   - El cliente, una vez recibida la notificación por parte del servidor, debe eliminar dicho fichero de su directorio de trabajo y cerrar la conexión con el servidor.

   Implemente dos versiones distintas: una con TCP y otra con UDP.

   La comunicación entre los procesos cliente/servidor debe realizarse utilizando únicamente sockets, no está permitido el uso de librerías de comunicación avanzadas.

Se deben utilizar ficheros binarios, en lugar de ficheros de texto, para poder enviar ficheros (.pdf) a través del socket correctamente.

Posible solución - opción 1

```
#Servidor TCP
import socket
import os

def archivo_existe(nombre_archivo):
   return os.path.isfile(nombre_archivo)

def recibir_archivo(conn, nombre_archivo):
   with open(nombre_archivo, 'wb') as archivo:
       while True:
           datos = conn.recv(1024)
           # Se detiene cuando el cliente cierra la conexión
           if not datos:
               break
           archivo.write(datos)

def iniciar_servidor(direccion, puerto):
   servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
   servidor.bind((direccion, puerto))
   servidor.listen(1)
   print(f"Servidor escuchando en {direccion}:{puerto}")

   while True:
       try:
           conn, addr = servidor.accept()
           print(f"Conexión establecida desde {addr}")

           # Recibir nombre del archivo
           nombre_archivo =
conn.recv(1024).decode('utf-8').strip()
           print(f"Cliente desea enviar: {nombre_archivo}")

           if archivo_existe(nombre_archivo):
               conn.sendall(b"EXISTE")
               respuesta = conn.recv(1024).decode('utf-8').strip()
               if respuesta != "SI":
                   print("El cliente canceló la transferencia.")
                   conn.close()
                   continue
           else:
               conn.sendall(b"OK")

           # Recibir el archivo hasta que el
           # cliente cierre la conexión
           recibir_archivo(conn, nombre_archivo)
           print(f"Archivo {nombre_archivo} recibido
correctamente.")
```

```python
        except Exception as e:
            print(f"Error en el servidor: {e}")

        finally:
            conn.close()

if __name__ == "__main__":
    iniciar_servidor('localhost', 65432)
```

```python
#Cliente TCP
import socket
import os

def enviar_archivo(servidor, puerto, ruta_archivo):
    cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:
        cliente.connect((servidor, puerto))

        nombre_archivo = os.path.basename(ruta_archivo)
        cliente.sendall(nombre_archivo.encode('utf-8'))

        respuesta = cliente.recv(1024).decode('utf-8').strip()
        if respuesta == "EXISTE":
            accion = input(f"El archivo {nombre_archivo} ya existe
en el servidor. ¿Desea sobrescribirlo? (SI/NO): ").strip().upper()
            cliente.sendall(accion.encode('utf-8'))
            if accion != "SI":
                print("Transferencia cancelada.")
                cliente.close()
                return

        # Enviar contenido del archivo
        with open(ruta_archivo, 'rb') as archivo:
            while True:
                datos = archivo.read(1024)
                if not datos:
                    # Finaliza cuando ya no hay más datos que leer
                    break
                cliente.sendall(datos)

        # Cerrar el socket completamente
        # para que el servidor detecte fin de transmisión
        cliente.close()

    except Exception as e:
        print(f"Error en el cliente: {e}")

if __name__ == "__main__":
    enviar_archivo('localhost', 65432, '/ruta/al/archivo.pdf')
```

Posible solución - opción 2

Control de bytes enviados y recibidos

```
#Servidor TCP
import socket
import os

def archivo_existe(nombre_archivo):
   return os.path.isfile(nombre_archivo)

def recibir_archivo(conn, nombre_archivo, tamano_archivo):
   with open(nombre_archivo, 'wb') as archivo:
       bytes_recibidos = 0
       while bytes_recibidos < tamano_archivo:
           # Ajustar tamaño restante
           datos = conn.recv(min(1024, tamano_archivo -
bytes_recibidos))
           if not datos:
               break
           archivo.write(datos)
           bytes_recibidos += len(datos)

def iniciar_servidor(direccion, puerto):
   servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
   servidor.bind((direccion, puerto))
   servidor.listen(1)
   print(f"Servidor escuchando en {direccion}:{puerto}")

   while True:
       try:
           conn, addr = servidor.accept()
           print(f"Conexión establecida desde {addr}")

           # Recibir nombre del archivo
           nombre_archivo =
conn.recv(1024).decode('utf-8').strip()
           print(f"Cliente desea enviar: {nombre_archivo}")

           if archivo_existe(nombre_archivo):
               conn.send(b"EXISTE")
               respuesta = conn.recv(1024).decode('utf-8').strip()
               if respuesta != "SI":
                   print("El cliente canceló la transferencia.")
                   conn.close()
                   continue
           else:
               conn.send(b"OK")

           # Recibir tamaño del archivo (10 bytes fijos)
           tamano_archivo =
int(conn.recv(10).decode('utf-8').strip())
           print(f"Tamaño del archivo: {tamano_archivo} bytes")
```

```
            recibir_archivo(conn, nombre_archivo, tamano_archivo)
            print(f"Archivo {nombre_archivo} recibido
correctamente.")

            conn.send(b"Recibido")

        except Exception as e:
            print(f"Error en el servidor: {e}")

        finally:
            conn.close()

if __name__ == "__main__":
    iniciar_servidor('localhost', 65432)
```

```
#Cliente TCP
import socket
import os

def enviar_archivo(servidor, puerto, ruta_archivo):
    cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:
        cliente.connect((servidor, puerto))

        nombre_archivo = os.path.basename(ruta_archivo)
        cliente.send(nombre_archivo.encode('utf-8'))

        respuesta = cliente.recv(1024).decode('utf-8').strip()
        if respuesta == "EXISTE":
            accion = input(f"El archivo {nombre_archivo} ya existe
en el servidor. ¿Desea sobrescribirlo? (SI/NO): ").strip().upper()
            cliente.send(accion.encode('utf-8'))
            if accion != "SI":
                print("Transferencia cancelada.")
                cliente.close()
                return

        # Obtener tamaño del archivo y enviarlo en
        # formato fijo de 10 caracteres
        # zfill(10) -> 10 caracteres, rellenando con ceros
        tamano_archivo =
str(os.path.getsize(ruta_archivo)).zfill(10)
cliente.send(tamano_archivo.encode('utf-8'))

        # Enviar contenido del archivo
        with open(ruta_archivo, 'rb') as archivo:
            bytes_enviados = 0
            while bytes_enviados < int(tamano_archivo):
                datos = archivo.read(1024)
                cliente.send(datos)
```

```
                bytes_enviados += len(datos)

        # Alternativamente, en TCP, podemos enviar todo
        # el fichero de una vez con sendall()
        # with open(ruta_archivo, 'rb') as archivo:
        #     cliente.sendall(archivo.read())

        # Esperar confirmación
        notificacion = cliente.recv(1024).decode('utf-8').strip()
        if notificacion == "Recibido":
            print("Servidor ha recibido el archivo. Eliminando
archivo local...")
            os.remove(ruta_archivo)

    except Exception as e:
        print(f"Error en el cliente: {e}")

    finally:
        cliente.close()

if __name__ == "__main__":
    enviar_archivo('localhost', 65432, '/ruta/al/archivo.pdf')
```

Posible solución - opción 3

Control de fin de fichero con <EOF>

```
# Servidor TCP
import socket
import os

def archivo_existe(nombre_archivo):
    return os.path.isfile(nombre_archivo)

def recibir_archivo(conn, nombre_archivo):
    with open(nombre_archivo, 'wb') as archivo:
        while True:
            datos = conn.recv(1024)
            if datos.endswith(b"<EOF>"):
                # Eliminar delimitador
                archivo.write(datos[:-5])
                break
            archivo.write(datos)

def iniciar_servidor(direccion, puerto):
    servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    servidor.bind((direccion, puerto))
    servidor.listen(1)
    print(f"Servidor escuchando en {direccion}:{puerto}")

    while True:
```

```python
        try:
            conn, addr = servidor.accept()
            print(f"Conexión establecida desde {addr}")

            nombre_archivo =
conn.recv(1024).decode('utf-8').strip()
            print(f"Cliente desea enviar: {nombre_archivo}")

            if archivo_existe(nombre_archivo):
                conn.send(b"EXISTE")
                respuesta = conn.recv(1024).decode('utf-8').strip()
                if respuesta != "SI":
                    print("El cliente canceló la transferencia.")
                    conn.close()
                    continue
            else:
                conn.send(b"OK")

            recibir_archivo(conn, nombre_archivo)
            print(f"Archivo {nombre_archivo} recibido.")
            conn.send(b"Recibido")

        except Exception as e:
            print(f"Error en el servidor: {e}")

        finally:
            conn.close()

if __name__ == "__main__":
   iniciar_servidor('localhost', 65432)
```

```python
# Cliente TCP
import socket
import os

def enviar_archivo(servidor, puerto, ruta_archivo):
   cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

   try:
       cliente.connect((servidor, puerto))

       nombre_archivo = os.path.basename(ruta_archivo)
       cliente.send(nombre_archivo.encode('utf-8'))

       respuesta = cliente.recv(1024).decode('utf-8').strip()
       if respuesta == "EXISTE":
           accion = input(f"El archivo {nombre_archivo} ya existe
en el servidor. ¿Desea sobrescribirlo? (SI/NO): ").strip().upper()
           cliente.send(accion.encode('utf-8'))
           if accion != "SI":
               print("Transferencia cancelada.")
               cliente.close()
```

```
                return

        with open(ruta_archivo, 'rb') as archivo:
            while True:
                datos = archivo.read(1024)
                if not datos:
                    break
                cliente.send(datos)

        # Enviar delimitador de fin de archivo
        cliente.send(b"<EOF>")

        notificacion = cliente.recv(1024).decode('utf-8').strip()
        if notificacion == "Recibido":
            print("Servidor ha recibido el archivo. Eliminando
archivo local...")
            os.remove(ruta_archivo)

    except Exception as e:
        print(f"Error en el cliente: {e}")

    finally:
        cliente.close()

if __name__ == "__main__":
    enviar_archivo('localhost', 65432, '/ruta/al/archivo.pdf')
```

Ejemplo de preguntas

```
#Servidor TCP
import ______________
import ______________

def archivo_existe(nombre_archivo):
    return os.path.isfile(nombre_archivo)

def recibir_archivo(conn, nombre_archivo):
    with open(nombre_archivo, ______________) as ______________:
        while True:
            datos = ______________
            if not datos:
                break
            archivo.write(datos)

def iniciar_servidor(______________, ______________):
    servidor = socket.socket(______________, ______________)
    servidor.bind((______________, ______________))
    servidor.listen(1)
    print(f"Servidor escuchando en {direccion}:{puerto}")
```

```python
    while True:
        try:
            conn, addr = ________________
            print(f"Conexión establecida desde {addr}")

            nombre_archivo = ________________
            print(f"Cliente desea enviar: {nombre_archivo}")

            if archivo_existe(nombre_archivo):
                conn.send(b"EXISTE")
                respuesta = ________________
                if respuesta != "SI":
                    print("El cliente canceló la transferencia.")
                    conn.close()
                    continue
            else:
                conn.________________

            recibir_archivo(conn, nombre_archivo)
            print(f"Archivo {nombre_archivo} recibido
correctamente.")

        except Exception as e:
            print(f"Error en el servidor: {e}")

        finally:
            conn.________________

if __name__ == "__main__":
    iniciar_servidor(________________, ________________)
```

```python
#Cliente TCP
import ________________
import ________________

def enviar_archivo(________________, ________________,
ruta_archivo):
    cliente = socket.socket(________________, ________________)

    try:
        cliente.connect((________________, ________________))

        nombre_archivo = os.path.basename(ruta_archivo)
        cliente.s________________

        respuesta = ________________
        if respuesta == "EXISTE":
            accion = input(f"El archivo {nombre_archivo} ya existe
en el servidor. ¿Desea sobrescribirlo? (SI/NO): ").strip().upper()
            cliente.send(accion.encode('utf-8'))
            if accion != "SI":
                print("Transferencia cancelada.")
```

```
                cliente._______________
                return

        # Enviar contenido del archivo
        with open(ruta_archivo, _______________) as _______________:
            while True:
                datos = _______________
                if not datos:
                    # Finaliza cuando ya no hay más datos que leer
                    break
                cliente._______________

        cliente._______________

    except Exception as e:
        print(f"Error en el cliente: {e}")

if __name__ == "__main__":
    enviar_archivo(__________, ___________, '/ruta/al/archivo.pdf')
```

3. Crear un programa cliente y un programa servidor en Python. El programa cliente enviará un fichero de texto al servidor, el servidor lo invertirá y lo devolverá al cliente junto con el tamaño del fichero.

- El programa cliente solicitará al usuario que introduzca el nombre del archivo.
- El cliente enviará el archivo (que debe existir) al servidor y mostrará por la terminal el mensaje `"Archivo enviado"`.
- El servidor invierte el contenido (texto) del archivo recibido, calcula su tamaño y envía ambos al cliente.
- El cliente debe recibir el archivo invertido y su tamaño. Además debe mostrar por pantalla el mensaje `"Recibido archivo invertido"`.

Ejemplo:

```
El usuario indica el nombre del archivo, por ejemplo, Hello.txt.

El archivo Hello.txt contiene el texto "Hello World!"

El cliente envía el fichero al servidor. El servidor lo invierte, calcula su tamaño y devuelve ambos al cliente.

El cliente recibe el archivo Hello.txt. En este ejemplo concreto, el texto que contiene el archivo invertido sería "!dlorW olleH" y el tamaño 12 bytes.
```

4. Escriba el código de un programa cliente y un programa servidor en Python, los cuales se comunicarán utilizando el protocolo UDP. El flujo de interacción debe aproximarse a lo siguiente:

   - El programa cliente establecerá la conexión y el programa servidor automáticamente le enviará el mensaje "`¡Bienvenido! ¿Cuál es su nombre para que pueda dirigirme a usted?`"
   - El cliente debe enviar un mensaje de texto con su nombre.
   - El servidor capturará el nombre indicado por el cliente y le dará una respuesta personalizada. Por ejemplo: "`nombre, ¿en qué puedo ayudarte?`"
   - El cliente debe enviar un mensaje de texto. Por ejemplo: "`¿Cuándo empiezan las clases este curso?`"
   - El servidor, llegados a este punto, siempre respondería lo mismo: "`Debe ponerse en contacto con el servicio de atención de dudas cuya dirección es dudas@ejemplo.com`"
   - Los puntos 4 y 5, de esta secuencia, se repiten hasta que el cliente se desconecta enviando la cadena de texto `exit` al servidor.

5. Crear un programa cliente y un programa servidor en Python. El programa servidor enviará una imagen de extensión .jpg al cliente, de acuerdo al siguiente flujo de interacción:

   - El programa cliente enviará al servidor el nombre de un fichero con extensión .jpg.
   - El servidor, recibida la petición del cliente, debe comprobar que el fichero existe. Si no es así, debe devolver un mensaje de error al cliente y finalizar.
   - Si el fichero existe, debe enviar el fichero al cliente, quien debe recibirlo y crear el fichero en su directorio de trabajo.

   Debe utilizarse TCP como protocolo de transporte.

   Tenga en cuenta que para poder enviar imágenes a través del socket correctamente, deben operarse como ficheros binarios, en vez de como ficheros de texto.

6. Crear un programa cliente y un programa servidor que se comunicarán mediante sockets UDP. El cliente podrá enviarle los siguientes comandos al servidor, los cuales podrá introducir el usuario a través del teclado:

   - `ls`: el servidor devolverá una lista con todos los ficheros de su directorio actual al cliente.

- `rm <nombre_fichero>`: se borrará el fichero indicado en el directorio del servidor.
- `write <nombre_fichero> '<mensaje>'`: el servidor creará un fichero llamado `<nombre_fichero>` y escribirá el texto `<mensaje>` dentro de él.
- `exit`: se cerrará la conexión y ambos programas finalizarán.
- `cd <nombre_directorio>`: el cliente cambia el directorio de trabajo en el servidor. Si el directorio especificado no existe, el servidor debe devolver un error al cliente.
- `mv <origen> <destino>`: el cliente mueve un fichero desde el directorio origen al directorio destino. Los directorios deben ser distintos. El servidor responderá al cliente indicándole si la operación se ha realizado correctamente, en caso contrario, el servidor debe devolver un error.

Después de cada comando, excepto para `ls`, el servidor responderá al cliente indicándole si la operación se ha realizado correctamente.

7. FTP simple. Escribir un programa cliente/servidor en el que el servidor recibe por socket el nombre de un fichero y se lo envía al cliente. El programa se puede mejorar, por ejemplo, enviando comandos como "listar", que listará los ficheros disponibles; enviando mensajes de error cuando no exista el fichero, permitiendo que el cliente también pueda subir sus propios ficheros, etc.

8. Realice un chat simple, en principio síncrono: un usuario habla, y se espera a que el otro usuario responda. Utilice la función de Python que existe para leer por teclado. Añada algún comando, como por ejemplo "desconectar" que desconecta al usuario y avisa al otro usuario para que cierre su conexión de forma segura.

   Implemente una versión con TCP y otra con UDP. ¿Qué diferencias hay entre ellas? Describa el flujo que ocurre entre los participantes.

Resumen

Funcionamiento de try y fynally en python

- **try**: Ejecuta el código que contiene y si durante la ejecución de este bloque de código ocurre una excepción, busca si hay un bloque except para manejar la excepción. Si no hay un bloque except pero sí un bloque finally, ejecutará el código en finally antes de propagar la excepción hacia arriba en la pila de llamadas.

- **finally**: Ejecuta siempre después del bloque try, independientemente de si ocurrió una excepción o no. Su propósito principal es permitir la ejecución de tareas de limpieza que deben realizarse bajo cualquier circunstancia, como cerrar archivos o liberar recursos de red como conexiones de socket.

Combinaciones comunes de permisos de acceso a archivos

- **'r+'**: Abre el archivo para lectura y escritura, posicionando el puntero al inicio del archivo.
- **'w+'**: Abrir un archivo para escritura y lectura. Sobrescribe el archivo si existe; crea un nuevo archivo si no existe.
- **'a+'**: Abrir un archivo para leer y añadir. Los datos escritos se añaden al final del archivo. Crea un nuevo archivo si no existe.
- **'rb'** o **'wb'**: Leer o escribir en modo binario. Útil para archivos que no son de texto (como imágenes o archivos ejecutables), y por tanto, los datos se tratan como bytes en lugar de cadenas de texto.

Consultar socket asignado y puerto sin hacer bind

El método **getsockname()** es un método de instancia en objetos de socket en Python, perteneciente al módulo socket. Este método es utilizado para obtener la dirección local del socket, es decir, la dirección IP y el puerto al que está vinculado el socket en el lado del host local. La necesidad de utilizar getsockname() surge en situaciones donde el sistema operativo asigna automáticamente un puerto local (y potencialmente una dirección IP) a un socket, especialmente cuando el socket es un cliente que no ha sido explícitamente vinculado a una dirección y puerto usando bind().

Ejemplo de uso:
```
print("Escuchando en:", nombreSocket.getsockname())
```

Ejemplo de salida:
Escuchando en: ('0.0.0.0', 56756)

Dirección IP '0.0.0.0': Normalmente, esta dirección se utiliza para denotar "todas las direcciones IPv4 en la máquina local". Esto suele significar que el sistema operativo ha asignado el socket a todas las interfaces de red disponibles para el envío de paquetes. No obstante, en algunos contextos o implementaciones específicas, ver '0.0.0.0' en la salida de getsockname() después de enviar un mensaje puede ser una peculiaridad de cómo el sistema operativo o la biblioteca de sockets están reportando la dirección de un socket que no ha sido explícitamente vinculado (a través de bind()) a una dirección IP específica.

Reutilizar dirección IP y puerto

El método **setsockopt** del módulo socket en Python permite modificar varias opciones del socket, y una de las opciones útiles en el desarrollo de aplicaciones de red es **SO_REUSEADDR**. Esta opción le dice al kernel que reutilice una dirección local en el caso de que aún esté en el estado TIME_WAIT, lo que puede ser particularmente útil para servidores que se reinician rápidamente y que necesitan volver a enlazarse a la misma dirección IP y puerto sin esperar que expire el tiempo de espera.

```python
# Ejemplo de servidor que incluye sotsockopt
import socket

# Configuración del host y el puerto
HOST = '127.0.0.1'
PORT = 12345

# Creación del socket TCP/IP
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Se establece la opción SO_REUSEADDR en 1
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# Enlace del socket al puerto
server_socket.bind((HOST, PORT))
print(f"Servidor iniciado en {HOST}:{PORT}. Esperando conexiones...")

# Escuchando conexiones entrantes
server_socket.listen()

# Aceptar una conexión
client_socket, client_address = server_socket.accept()
print(f"Conexión aceptada desde {client_address}.")

# Cerrar los sockets
client_socket.close()
server_socket.close()
```

```python
# Ejemplo de cliente que incluye sotsockopt
import socket

# Configuración del servidor al que se desea conectar
SERVER_HOST = '127.0.0.1'
SERVER_PORT = 12345

# Creación del socket TCP/IP
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```python
# Establecimiento de la opción SO_REUSEADDR en 1
client_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# Enlace del socket a un puerto específico
# Esto es opcional y solo necesario si se necesita que el cliente use un
puerto específico
CLIENT_PORT = 54321 # Ejemplo de puerto cliente; en la práctica, podría
ser cualquier puerto no utilizado

client_socket.bind(('', CLIENT_PORT)) # Se enlaza a cualquier dirección
IP del cliente y a un puerto específico

# Conexión al servidor
client_socket.connect((SERVER_HOST, SERVER_PORT))

# Imprimir por pantalla IP y puerto del servidor
print(f"Conectado al servidor {SERVER_HOST}:{SERVER_PORT}")

# Envío y recepción de datos...

# Cierre del socket
client_socket.close()
```